

Weather Info Dashboard

Student Name: Hiba Shaukat

Assignment: Weather Info Dashboard

Tech Stack: React, Vite, JavaScript, OpenWeatherMap API

Submission Date: June 2026

1. Project Overview

WeatherLens is a React-based frontend application that fetches and displays real-time weather data using the OpenWeatherMap API. The primary objective of this assignment was to implement proper state management for three distinct UI states: Loading, Error, and Success.

The application allows users to search any city worldwide and instantly view current weather conditions including temperature, humidity, wind speed, pressure, and visibility — all within a clean, modern interface.

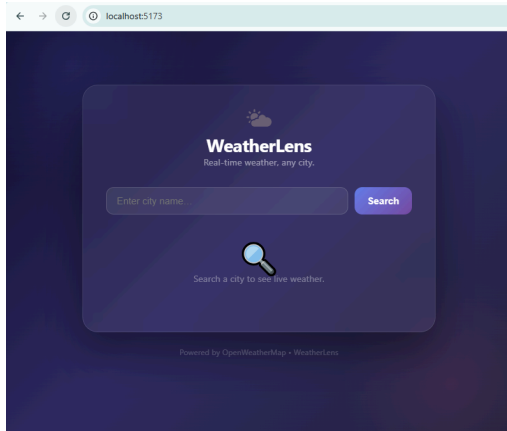
2. Application Features

- City-based weather search with Enter key and button support
 - Real-time data from OpenWeatherMap free tier API
 - Three fully handled UI states (Loading, Error, Success)
 - Weather details: temperature, feels-like, humidity, wind speed & direction, pressure, visibility, cloud cover, min/max temperature
 - Weather condition emoji mapped from API condition codes
 - Responsive design compatible with mobile and desktop
-

3. Screenshots — All Application States

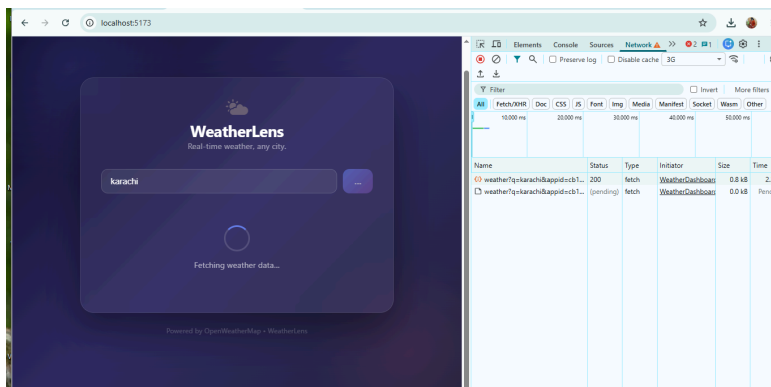
3.1 Initial / Empty State

This is the default view when the app first loads, before any search is performed.



3.2 Loading State

When the user clicks Search, the app immediately enters a loading state. The spinner appears, the Search button is disabled, and the previous result is cleared.

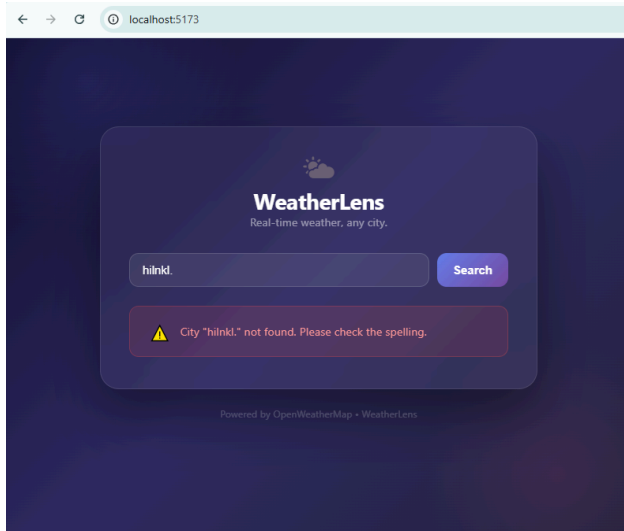


How it works in code:

```
js
setLoading(true); // triggers spinner
setError(null);   // clears previous errors
setWeather(null); // clears previous result
```

3.3 Error State — City Not Found (404)

When an invalid or misspelled city name is entered, the API returns a 404 response. The app catches this and displays a specific, user-friendly error message.



How it works in code:

```
js
if (res.status === 404) {
  throw new Error(`City "${city}" not found. Please check the spelling.`);
}
```

3.4 Error State — Network Failure

If the user has no internet connection, the fetch call throws a `TypeError`. The app catches this separately and shows a network-specific message.

How it works in code:

```
js
if (err.name === "TypeError") {
  setError("Network error. Please check your internet connection.");
}
```

3.5 Error State — API Limit Exceeded (429)

If the free API quota is exceeded, the server responds with a 429 status. The app handles this distinctly from other errors.

How it works in code:

```
js
if (res.status === 429) {
  throw new Error("API limit exceeded. Please wait a moment and try again.");
}
```

```
}
```

3.6 Success State

When a valid city is entered and the API responds with 200 OK, the full weather card is displayed with all available data.



How it works in code:

```
js
const data = await res.json();
setWeather(data); // triggers the weather card to render
```

4. Code Structure

File / Section	Purpose
<code>WeatherDashboard.jsx</code>	Main component — handles state, API call, and rendering
<code>StatTile</code>	Reusable sub-component for each weather detail tile
<code>API_KEY / API_BASE</code>	Configuration constants defined at the top of the file
<code>getWeatherEmoji(code)</code>	Maps OWM condition codes to relevant emoji

<code>getWindDir(deg)</code>	Converts wind degree value to compass direction
<code>styles</code> object	All component styles defined in one place (CSS-in-JS)

5. Error Handling Summary

Error Type	HTTP Status	Message Shown to User
City not found	404	"City not found. Please check the spelling."
Invalid API key	401	"Invalid API key. Please check your OpenWeatherMap API key."
Rate limit exceeded	429	"API limit exceeded. Please wait a moment and try again."
Network failure	TypeError	"Network error. Please check your internet connection."
Other errors	Any other	"Something went wrong (Error X). Try again later."

6. Conclusion

This assignment demonstrated the importance of handling all possible states in a frontend application. A real-world app cannot simply show data — it must gracefully manage delays, failures, and edge cases. WeatherLens successfully implements all three states with clear visual feedback for the user, making the experience intuitive and reliable.